

Rapport de Stage

Dufour Marc

Actemium Process Control Le Havre

Du 12 Mai au 13 Juin

Remerciements

Actemium :

Mr Charles Dehais, pour m'avoir aidé à rentrer en contact avec Actemium;

Mme Pascale Brohan pour m'avoir accepter dans leur équipe et offert cette expérience;

Mrs Anthony Lambour développeur et tuteur de mon stage, et Thibaud Autrive, directeur technique de l'équipe de développement, pour m'avoir accompagné et dirigé tout au long de ce stage et avoir répondu à tous mes questionnements;

Mrs Fabrice Trotel, Fleurian Moal, Thibaud Tailly et Philippe Saforge, membres de l'équipe de développement pour m'avoir aidé tout au long du projet sur des blocages et des questions.

A l'équipe entière d'Actemium Process Control Le Havre, avec qui je n'ai pas forcément eu l'occasion de travailler, mais qui m'ont merveilleusement accueilli dans leur locaux et dans leur équipe sur toute la durée de ce stage.

St Jo Sup :

Mr Nicolas Lecordier et Mme Magali André, professeur.e à l'institut St Jo Sup, pour leur aide, leur conseil et leur soutien tout au long de l'année scolaire.

Autres :

A mon père, Franck Dufour, qui m'a poussé à aller chercher en dehors du Havre, et sans qui je n'aurait jamais pu effectuer ce stage dans un environnement aussi bienveillant.

Sommaire :

Remerciements	2
Actemium :	2
St Jo Sup :	2
Autres :	2
Sommaire :	3
Glossaire :	4
Solution full web :	4
ERP :	4
SAP :	4
Lecteur RFID :	4
API :	4
EDI :	4
WorkFlow :	4
DAL (Data Access Layer) :	5
BLL (Business Logic Layer) :	5
API (Controllers) :	5
DTO :	5
Endpoint API :	5
Endpoint HTTP GET :	5
Présentations :	6
L'entreprise : Actemium	6
L'équipe de développement :	6
Contenu du stage :	7
GPAC, projet globalisé :	7
Fonctionnalités principales par module	7
Points forts	8
Cas d'usage typique	9
Clients et secteurs	9
Mon projet : Notification Service	10
Description et objectif :	10
Organisation du projet :	10
Architecture du projet :	11
Chronologie du projet :	13
Semaine 1 :	13
Semaine 2 :	16
Semaine 3 :	17
Semaine 4 :	19
Semaine 5 :	21
Semaine 6 :	23
Conclusion :	25

Glossaire :

Solution full web :

Une **solution full web** est un logiciel accessible directement depuis un navigateur internet, sans installation sur les postes utilisateurs. Elle permet une utilisation simple, depuis n'importe quel appareil connecté, avec des mises à jour centralisées et une gestion sécurisée des données.

ERP :

Un **ERP (Enterprise Resource Planning)** est un logiciel qui centralise et gère l'ensemble des processus d'une entreprise (comptabilité, logistique, production, RH...). Dans le contexte d'une interface, il sert à **échanger automatiquement des données** entre le système industriel (comme GPAC) et l'entreprise (commandes, stocks, livraisons, etc.).

SAP :

SAP est un logiciel de gestion d'entreprise très utilisé, qui centralise des fonctions comme la comptabilité, les achats, la logistique ou la production. L'interface permet de **connecter SAP à un autre système** (comme GPAC), pour échanger automatiquement des données : par exemple, envoyer une commande depuis SAP vers GPAC, ou remonter un bon de livraison depuis GPAC vers SAP.

Lecteur RFID :

Un **lecteur RFID** est un appareil qui lit les informations contenues dans des badges ou étiquettes RFID (à puce sans contact). Dans une interface industrielle, il permet par exemple d'**identifier automatiquement un véhicule, un opérateur ou un produit** lors de son passage.

API :

API (Application Programming Interface) : c'est une interface logicielle qui permet à deux systèmes de **communiquer automatiquement en temps réel**, comme l'envoi ou la réception de données (ex. : stock, commandes).

EDI :

EDI (Échange de Données Informatisé) : c'est un standard d'échange de documents (factures, bons de commande...) **entre entreprises**, souvent utilisé dans la logistique, de façon plus structurée mais moins flexible que l'API.

Workflow :

Un **workflow** est une suite d'étapes ou d'actions organisées de manière logique pour **réaliser un processus** (par exemple : validation d'une commande, chargement d'un camion). Il permet d'automatiser, suivre et contrôler les tâches entre différents acteurs ou systèmes.

DAL (Data Access Layer) :

La **DAL** (couche d'accès aux données) gère toutes les opérations liées à la base de données. Elle centralise les requêtes pour **lire, insérer, modifier ou supprimer des informations**. Elle permet de séparer l'accès aux données du reste de l'application. Ainsi, elle facilite la maintenance et améliore la sécurité des données.

BLL (Business Logic Layer) :

La **BLL** (couche de logique métier) contient toutes les règles de gestion du projet. Elle **définit comment traiter les données** selon les besoins fonctionnels. Elle sert d'intermédiaire entre la [DAL](#) et l'[API](#). Son rôle est de garantir que les règles métier soient appliquées de manière cohérente.

API (Controllers) :

Les Controllers constituent l'API de l'application. Ils **reçoivent les requêtes** des utilisateurs ou d'autres systèmes. Ils appellent la [BLL](#) pour traiter les données et renvoient une réponse (souvent en JSON). Ils assurent le bon fonctionnement de la communication entre l'application et l'extérieur.

DTO :

Les **DTO (Data Transfer Objects)** sont des objets structurés servant à transférer des données spécifiques entre les couches d'une application ([API](#), [BLL](#), [DAL](#)) ou entre serveur et client. Ils encapsulent uniquement les champs nécessaires pour une opération donnée, permettant de découpler le modèle de données interne des interfaces externes. Cette approche améliore la sécurité, facilite la validation des données et permet d'adapter le format sans modifier la base de données.

Endpoint API :

Un **endpoint API** est une URL exposée par une application permettant à des clients (comme une interface web ou mobile) d'accéder à une ressource ou d'exécuter une action via une requête HTTP (GET, POST, etc.). C'est le **point d'entrée** d'une API pour envoyer ou recevoir des données.

Endpoint HTTP GET :

Un **endpoint HTTP GET** est une adresse à laquelle une application peut envoyer une demande pour **obtenir des informations** (comme une liste ou un détail). C'est une façon simple pour un site ou une appli de poser une question à un serveur et de recevoir une réponse.

Présentations :

L'entreprise : Actemium

L'entreprise où j'ai effectué mon stage, Actemium Process Control, fait partie depuis 2009 du groupe [Vinci Energies](#), d'abord sous le nom Cegelec, puis renommé en 2014 sous le nom Actemium, qu'elle garde encore aujourd'hui.

[Actemium](#) accompagne les industriels dans l'optimisation de leurs installations tout au long de leur cycle de vie. Ses activités couvrent l'électricité industrielle, l'automatisation, la maintenance, l'ingénierie de process et la digitalisation (Industrie 4.0). Présente dans plus de 40 pays avec 400 unités opérationnelles, elle intervient dans des secteurs variés comme l'agroalimentaire, l'automobile, l'énergie, la chimie ou la pharmacie. Actemium se distingue par une approche locale et une expertise sectorielle forte.

La division Actemium Process Control automatise et pilote les procédés industriels (automates, supervision, régulation). Elle optimise la production, améliore la qualité et sécurise les installations dans des secteurs comme la chimie, l'agroalimentaire ou la pharmaceutique.

L'équipe de développement :



ORGANIGRAMME

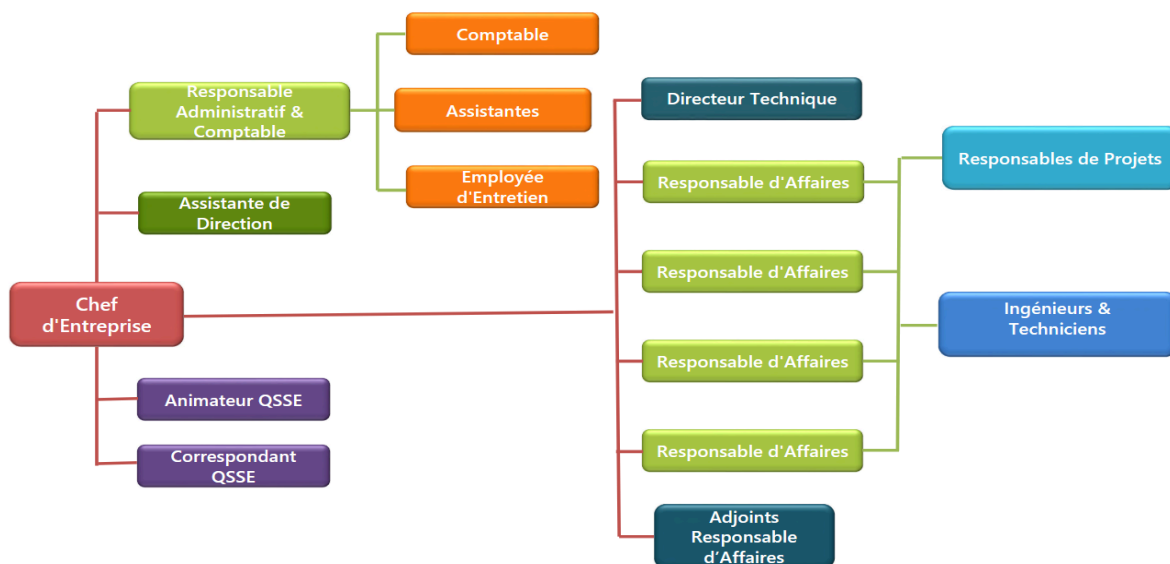


Image 1 : Organigramme non nominatif d'Actemium Process Control

Durant la durée de mon stage, j'ai eu l'occasion de rencontrer et d'échanger avec plusieurs membres de l'équipe Actemium Process Control, qui m'ont accompagné tout au long de mon projet.

Monsieur **Thibaut Autrive**, directeur technique, a été mon principal interlocuteur pour le cadrage du projet. C'est lui qui m'a présenté les objectifs, les enjeux fonctionnels et les attentes en termes de qualité et de méthodologie. Son expertise m'a permis de mieux comprendre les contraintes spécifiques liées aux environnements industriels et de progresser dans la définition des priorités techniques.

L'équipe de développement, avec laquelle j'ai collaboré régulièrement, est composée de :

- **Fabrice Trotel**
- **Fleurian Moal**
- **Thibaud Tailly**
- **Philippe Saforge**

Ces collaborateurs ont su se montrer disponibles pour répondre à mes questions, m'orienter dans la prise en main des outils, et m'apporter leur soutien lors des phases plus complexes du développement. Leurs conseils ont été essentiels pour avancer efficacement et gagner en autonomie.

Enfin, Monsieur **Charles Dehais**, responsable chargé d'affaires, a également participé à mon intégration au sein de l'entreprise et m'a permis de mieux comprendre le contexte global des projets menés par Actemium Process Control.

Grâce à l'accompagnement de ces professionnels, j'ai pu bénéficier d'un encadrement de qualité et m'impliquer pleinement dans la réalisation de mon projet.

Contenu du stage :

GPAC, projet globalisé :

Solution 100 % web : accessible via navigateur, entièrement configurable selon les **GPAC (Gestion des Postes Automatisés de Chargement)** est un **progiciel métier développé par Actemium Process Control**. Il est dédié à la **gestion automatisée et sécurisée des opérations logistiques** sur les sites industriels (dépôts pétroliers, usines chimiques, agroalimentaires, etc.).

C'est une [solution full web](#), modulaire, hautement personnalisable, conçue pour s'adapter aux processus spécifiques de chaque site client.

Fonctionnalités principales par module

1. Supervision

- Contrôle en temps réel des installations (pompes, vannes, bacs, postes de chargement)
- Supervision des niveaux de cuves, états des lignes, alarmes
- Gestion de la sécurité incendie et de l'interverrouillage
- Visualisation graphique des installations (synoptiques)

2. Gestion

- Planification des opérations de chargement (camions, wagons, etc.)
- Identification des véhicules et des opérateurs (lecteurs de badges, QR code)
- Suivi de stock (entrées/sorties, volumes disponibles, bilans matière)
- Traçabilité complète des produits, temps de chargement, incidents
- Génération automatique de documents (bons de livraison, documents douaniers, certificats, etc.)
- Gestion des accès et de la circulation sur site

3. Communication

- Interfaces avec :
 - Automates industriels (Siemens, Schneider, etc.)
 - [ERP](#) ([SAP](#), Oracle...)
 - Bases de données (SQL, Oracle)
 - Systèmes de pesée, [lecteurs RFID](#), caméras, capteurs de sécurité
 - Services douaniers ou autorités de régulation (e.g., via [API](#) ou [EDI](#))

Points forts

- **Modularité** : GPAC peut être déployé en partie ou en totalité, selon les besoins (supervision seule, gestion sans automatisme, etc.)
- **Personnalisation** : chaque interface, [workflow](#) ou règle métier peut être ajusté au fonctionnement du site client
- **Haute disponibilité** : support 24/7, télémaintenance, mises à jour planifiées sans interruption
- **Cybersécurité** : architecture sécurisée, gestion fine des droits utilisateurs, traçabilité complète des actions
- **Interface conviviale** : ergonomie pensée pour les opérateurs de terrain et les administrateurs

Cas d'usage typique

Un site pétrochimique utilise GPAC pour :

- Planifier l'arrivée de camions citernes
- Contrôler l'accès au site via badges
- Automatiser le remplissage des cuves via supervision automatisée
- Assurer la traçabilité complète du produit (du bac au camion)
- Générer les documents de livraison et douaniers automatiquement
- Envoyer les données au siège via une interface [SAP](#)

Clients et secteurs

Des entreprises comme **Total Energies**, **Rubis Terminal**, **Petroineos**, **Afriquia Gaz**, ou **Altens** utilisent GPAC pour gérer leurs sites industriels. Il est principalement déployé dans les secteurs :

- Pétrole et gaz
- Chimie
- Agroalimentaire
- Transport de matières dangereuses
- Énergie

Mon projet : Notification Service

Description et objectif :

Durant mes 6 semaines de stage, j'ai eu l'opportunité de relever un défi technique stimulant : l'intégration d'un système de notifications intelligent dans l'application de l'entreprise. Face à un temps imparti restreint, j'ai pu m'appuyer sur une documentation solide élaborée en amont par Thibaud Taily. Ce document détaille avec précision l'architecture, le fonctionnement et les cas d'usage de l'application, me permettant ainsi d'orienter mes efforts vers une intégration efficace et cohérente.

Le système de notifications que j'ai contribué à développer se déploie en deux volets complémentaires. Premièrement, au niveau de l'interface utilisateur, une icône interactive est visible sur l'écran principal. Dès qu'elle est cliquée, cette icône déclenche l'affichage d'une liste de notifications en temps réel. Chaque notification représente une opération critique – par exemple, l'arrivée d'un camion sur le site, le démarrage ou la finalisation d'un chargement, ou encore la mise en évidence d'un quai indisponible. L'algorithme de tri que j'ai mis en place s'assure que seule les informations les plus pertinentes, en fonction de leur durée de validité, soit présentée à l'utilisateur.

Mon travail s'est articulé autour de deux axes principaux : avancer de manière significative dans le développement du projet et documenter chaque étape avec minutie. J'ai ainsi consigné les choix techniques, les ajustements en cours de route et les commentaires de code afin de garantir une passation optimale. Cette démarche permet à toute personne reprenant le projet de comprendre rapidement le chemin parcouru et d'intervenir sur d'éventuelles erreurs, facilitant ainsi la continuité du développement par la suite.

Organisation du projet :

Compte tenu de la durée relativement courte de mon stage, une organisation rigoureuse s'est révélée indispensable pour garantir l'efficacité du travail accompli.

Dans cette optique, mon tuteur, **M. Anthony Lambour**, ainsi que le directeur technique, **M. Thibaud Autrive**, ont pris soin de structurer le projet en plusieurs tâches clairement définies, que je détaillerai dans les sections suivantes. Cette méthodologie a permis d'effectuer des points d'avancement réguliers avec mes encadrants, favorisant ainsi la détection et la correction rapides d'éventuelles erreurs ou écarts par rapport aux objectifs initiaux.

L'intégration au sein de l'entreprise impliquait également le respect de normes internes, notamment en matière d'architecture des fichiers, de choix des outils et de bonnes pratiques de développement. Ces règles ont constitué un cadre rassurant pour mener à bien mon travail tout en m'alignant sur les standards de l'équipe.

Afin de faciliter mon apprentissage et de me rendre rapidement opérationnel, il m'a été confié un projet déjà en cours de développement, accompagné des outils et ressources nécessaires à sa compréhension. Cette immersion progressive m'a permis d'acquérir une vision concrète des différentes couches logicielles et de m'approprier leur fonctionnement pour mieux les réutiliser dans mon propre projet.

Pour assurer un suivi efficace de l'avancement, **Microsoft Teams** a été utilisé comme principal outil de communication au sein de l'équipe. Par ailleurs, **GitLab** a permis d'organiser le projet en tâches distinctes et de centraliser le code afin que chaque modification soit versionnée et validée. Bien que j'aie été seul en charge de ce projet, l'utilisation de ces outils a contribué à maintenir un haut niveau de qualité, à vérifier régulièrement les évolutions du code et à garantir une progression conforme aux attentes.

Ainsi, grâce à cette méthodologie de travail structurée et à l'utilisation d'outils adaptés, j'ai pu progresser de manière autonome tout en bénéficiant d'un suivi régulier.

Cette organisation m'a permis d'appréhender efficacement les différentes composantes techniques et fonctionnelles du projet.

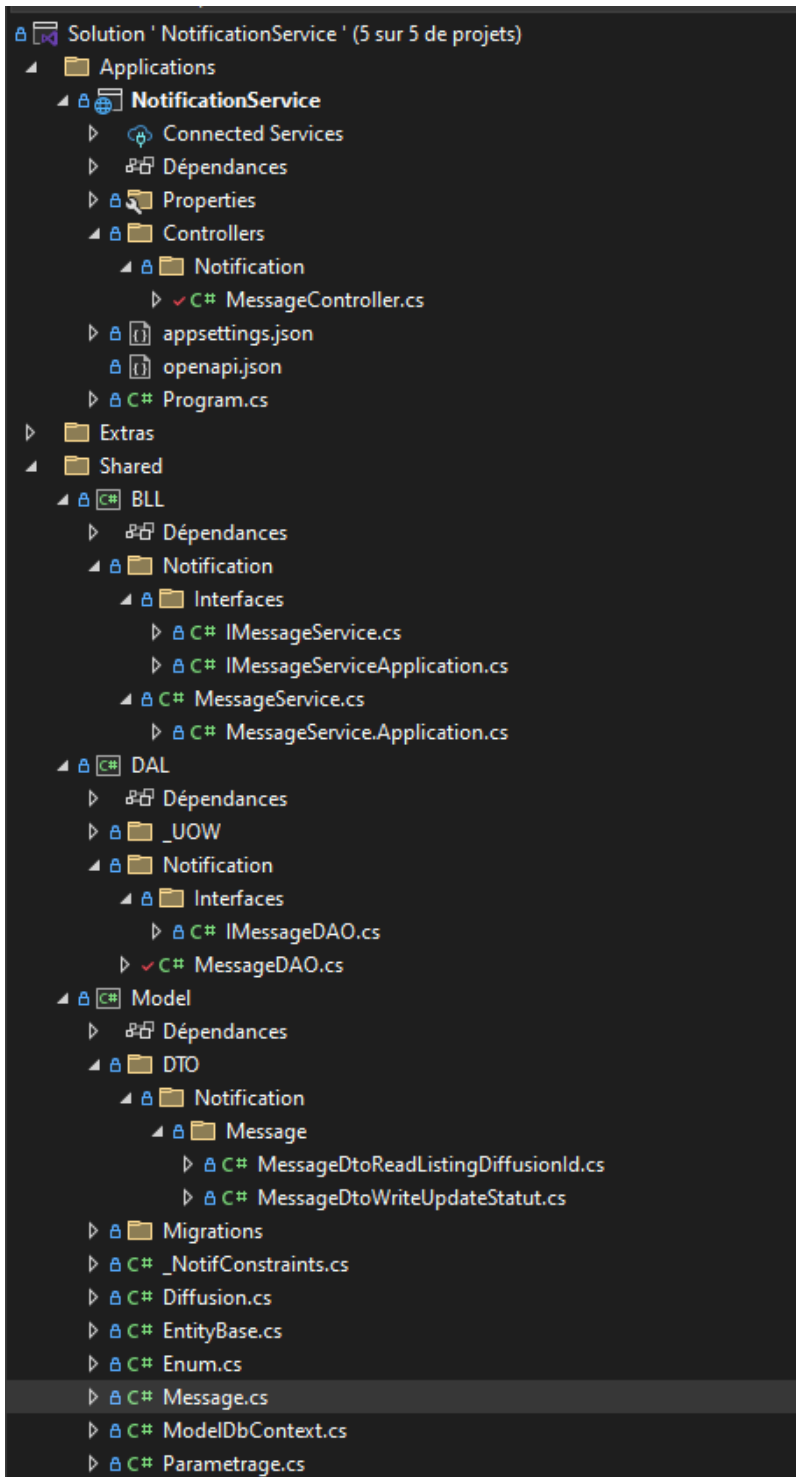
Dans la partie suivante, je présenterai plus en détail **l'architecture mise en place**, en décrivant les principaux éléments qui la composent ainsi que leur rôle au sein de l'application.

Architecture du projet :

Tableau 1 : Architecture du projet

Couche	Rôle	Fichiers concernés
Model	Définit les structures de données (Message, Diffusion, Parametrage) et le contexte de base de données via Entity Framework	Message.cs, Diffusion.cs, Parametrage.cs, ModelDbContext.cs
DAL (Data Access Layer)	Gère l'accès aux données de manière optimisée et sécurisée grâce au modèle Repository et au Unit of Work	MessageDAO.cs, UnitOfWork.cs
BLL (Business Logic Layer)	Regroupe la logique métier : traitement des données, application des règles spécifiques, validation	MessageService.cs, MessageService.Application.cs
API (Controllers)	Met à disposition des endpoints HTTP pour interagir avec le système	MessageController.cs
SQL / Extras	Contient les scripts SQL pour la création et l'initialisation de la base de données	scriptCrea.sql, ScriptInitTest.sql
Interfaces	Définissent des contrats que les classes doivent respecter. Elles facilitent l'organisation du code, la réutilisabilité et les tests.	IMessageService.cs, IMessageServiceApplication.cs, IMessageDAO.cs

Pour une représentation plus visuelle, voici une capture d'écran du projet, ouvert dans l'outil Visual Studio 2022 :



Pour les fichiers supplémentaires, nous avons :

Enum.cs : Regroupent des **valeurs constantes nommées** pour éviter les erreurs et rendre le code plus lisible (ex. états d'un message).

NotifConstraints.cs : Contient des **paramètres fixes** (valeurs par défaut, limites) pour centraliser la configuration de l'application.

EntityBase.cs : Classe **mère** commune aux entités, elle factorise des propriétés partagées (ex. Id, dates) pour éviter les répétitions.

Program.cs : **Point d'entrée** de l'application : configure et démarre l'exécution du projet, initialise le serveur et les services.

MessageDtoReadListingDiffusionId.cs et MessageDtoWriteUpdateStatut.cs : DTO permettant au contrôleur d'avoir accès uniquement aux informations nécessaires.

Image 2 : Architecture du projet dans la solution

Ces fichiers organisent le projet de façon claire et modulaire, ce qui facilite sa maintenance, son évolution et sa compréhension.

Chronologie du projet :

Semaine 1 :

La première semaine avait pour finalité de prendre place dans l'entreprise, me présenter à l'équipe et vis versa.

Une fois réalisé, et les équipements récupérés et préparés, j'ai pu commencer à travailler sur la mise en place de la base de données, fondations du projet, avec l'aide des recherches faites au préalable par Mr Thibaud Tailly.

Après quelque jours durant lesquelles j'ai cherché comment faire cela, et également retravailler le modèle et renommer quelques variables, nous sommes finalement tombés d'accord sur un modèle à 3 tables, dont voici le schéma :

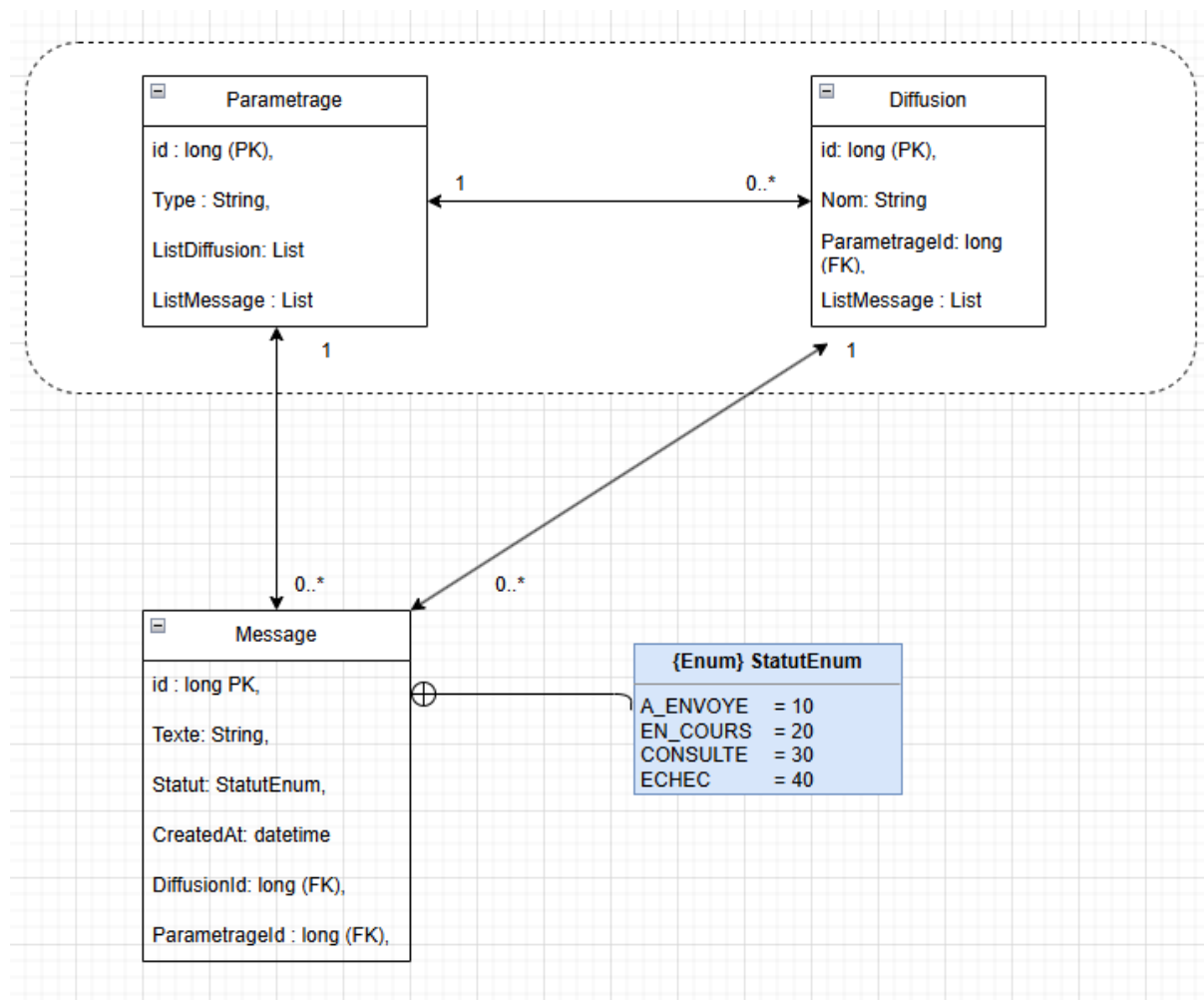


Image 3 : Modèle Entité-Associations du projet

Détails des 3 entités :

Message

Rôle

C'est l'entité principale — elle représente **un message de notification** que tu veux envoyer ou diffuser à des utilisateurs.

Attributs

- **Id** → identifiant unique du message (clé primaire)
- **Texte** → le texte ou le corps de la notification
- **Statut** → l'état du message (**A_ENVOYER**, **EN_COURS**, **CONSULTE**, **ECHEC**)
- **CreatedAt** → date et heure de création
- **ParametrageId** → identifiant externe d'un Parametrage utilisé pour le message
- **Parametrage** → Parametrage passer en attributs en plus de son identifiant. Permet d'éviter les recherches par identifiants dans les requêtes de Base de Données
- **DiffusionId** → identifiant externe d'un Diffusion utilisé pour le message
- **Diffusion** → Diffusion passer en attributs en plus de son identifiant. Permet d'éviter les recherches par identifiants dans les requêtes de Base de Données

Table SQL

Elle se traduit en table **Messages** en base de données.
Chaque ligne = un message unique.

Diffusion

Rôle

C'est une entité qui gère la **diffusion d'un message** :

- **À quel public cible** (Postes, personnes, profils)

Elle peut aussi servir à garder une **trace** : par exemple, on peut savoir qu'un message a été envoyé à 500 utilisateurs.

Attributs probables

- **Id** → identifiant unique de la diffusion
- **Nom** → nom du groupe de diffusion
- **ParametrageId** → identifiant externe d'un Parametrage utilisé pour le message
- **Parametrage** → Parametrage passer en attributs en plus de son identifiant. Permet d'éviter les recherches par identifiants dans les requêtes de Base de Données
- **ListMessage** → Liste des Message envoyé à ce groupe de diffusion

Table SQL

Table **Diffusions** — une diffusion est liée à **un seul message**, mais un message peut avoir **plusieurs diffusions** (ex. : envoyer le même message via mail + push).

Parametrage

Rôle

C'est une entité pour stocker les **paramètres de configuration** liés aux notifications :

- Configure le code attribuer à la notification à envoyer, prédéfini par le client.

Elle permet de rendre le système plus **souple** sans changer le code : il suffit de modifier une valeur en base.

Attributs

- **Id** → identifiant unique
- **Type** → Code de la notification, indiquant le type d'opération concernée
- **Durée** → temps de validité de la notification, exprimé en minutes
- **ListDiffusion** → List des groupes de diffusion auquel ce paramétrage est attribué
- **ListMessage** → List des notifications envoyé à ce paramétrage

Table SQL

Table **Parametrages** — elle peut contenir plusieurs paramètres généraux pour piloter la logique métier de diffusion.

Ces trois tables constituent la base solide de tout le projet. Consacrer une semaine à leur conception peut sembler long pour une quantité de code limitée, mais c'est un investissement essentiel : cela garantit une structure claire, stable et sans ambiguïté, évitant ainsi des révisions ou des blocages plus tard. Cette étape pose des fondations robustes pour construire la suite sereinement et efficacement.

Semaine 2 :

Durant cette deuxième semaine, l'objectif principal était de mettre en place l'environnement nécessaire au développement de l'application, ainsi que de configurer la base de données et ses interactions avec le backend.

1. Création de l'environnement Docker avec SQL Server

La première étape a consisté à créer un environnement de développement basé sur **Docker**, afin de garantir une infrastructure isolée, stable et facilement reproductible.

Un **conteneur SQL Server** a été mis en place à l'aide d'une image officielle, configurée avec les paramètres de connexion appropriés (nom de l'instance, mot de passe administrateur, ports, etc.).

Cela permet de disposer d'un moteur de base de données fonctionnel sans avoir besoin d'une installation locale complexe, tout en facilitant les tests et le déploiement.

2. Mise en place des migrations avec Entity Framework Core

J'ai ensuite configuré **Entity Framework Core**, l'ORM utilisé dans le projet, pour automatiser la gestion du schéma de base de données.

Les premières **migrations** ont été générées à partir des modèles C# définis, puis appliquées pour créer les tables correspondantes dans la base SQL Server. Cette approche permet une évolution maîtrisée de la structure de la base de données au fil du développement.

3. Connexion et vérification via SQL Server Management Studio

Une fois la base de données opérationnelle, j'ai utilisé **SQL Server Management Studio (SSMS)** pour :

- Vérifier la bonne création des tables et relations générées par les migrations,
- Tester des requêtes SQL manuellement,
- Contrôler la communication entre l'application et la base,
- Faciliter l'exploration et le débogage au besoin.

Semaine 3 :

Durant cette troisième semaine, j'ai développé une nouvelle fonctionnalité backend : la mise en place d'un **endpoint API** permettant de récupérer dynamiquement les messages associés à une **liste de diffusion**, tout en filtrant selon des critères métier précis.

1. Objectif fonctionnel

L'API devait permettre de **retourner tous les messages liés à une liste de diffusion donnée**, identifiée par son **ID**.

Cependant, seuls les **messages valides** devaient être renvoyés, selon deux conditions :

- Le message doit avoir une **durée de diffusion encore valide** (non expirée),
- Son **statut** doit être égal à **EN_COURS**.

Ce filtrage permet d'assurer que seuls les messages réellement actifs et pertinents sont exploités côté client.

2. Architecture et découpage technique

Pour respecter l'architecture en couches (clean architecture / DDD), cette fonctionnalité a été construite en suivant un découpage clair :

a) Controller (API Endpoint)

Création d'un **endpoint HTTP GET** exposé via le contrôleur.

Il reçoit un **ID** en paramètre et délègue la logique métier à un service dédié.

```
// *****
#region READ

/// <summary>
/// Resors une List de message en fonction d'un identifiant de Diffusion
/// </summary>
/// <param name="id"> Id de Diffusion </param>
/// [HttpGet("{id}")]
public async Task<ActionResult<List<MessageDtoReadListingDiffusionId>>> GetListMessageByDiffusionId([FromRoute, Required] long diffusionId, CancellationToken cancellationToken)
{
    try
    {
        // Requête :
        List<Message> listMessage = await _MessageServiceApplication.ReadListMessageByDiffusionId(diffusionId, cancellationToken);

        // Check des le statut des messages d'une list est bien fixé à "en_cours" en prenant
        List<Message> listMessageChecked = await _MessageServiceApplication.CheckStatutToEnCours(listMessage, cancellationToken);

        // Check la validité d'une liste de Message en comparant l'addition de la date de création et de la durée de validité au temps actuel
        listMessageChecked = await _MessageServiceApplication.SortValiditeDuree(listMessage, cancellationToken);

        //Map :
        IEnumerable<MessageDtoReadListingDiffusionId> listMessageDto = listMessageChecked.Select(MessageDtoReadListingDiffusionId.Map);

        // Réponse :
        return Ok(listMessageChecked);
    }
    catch (Exception exception)
    {
        return null;
    }
}

#endregion READ
// *****
```

Image 4 : Code de l'endpoint "GetListMessageByDiffusionId"

L'**ID** récupéré par la requête est utilisé pour sortir la liste brute. Nous vérifions ensuite si le statut est bien noté à "**EN_COURS**", et que la validité de la notification est bonne. Enfin, nous ressortons la liste vérifiée.

b) Service (Logique Métier)

Dans le **service MessageService**, la méthode **GetListMessageByDiffusionId** est chargée :

- d'appeler le DAO pour récupérer les messages liés à la liste,
- de filtrer les résultats en vérifiant la validité de la durée (ex. : date de fin > date actuelle),
- de ne conserver que ceux avec le statut **EN_COURS**.

Cette couche permet de centraliser les règles métier tout en restant indépendante des détails techniques.

```
public Task<List<Message>> ReadListMessageByDiffusionId(long diffusionId, CancellationToken cancellationToken)
=> _uow.MessageDAO.FindListMessageByDiffusionId(diffusionId, cancellationToken);
```

Image 5 : Code du service permettant de lier la partie métier à la DAO

c) DAO (Data Access Object)

Le **DAO MessageDao** contient la logique d'accès aux données.

Il interroge la base via Entity Framework en utilisant une jointure sur les tables **Message** et **ListeDiffusion**.

```
/// <summary>
/// Liste des Message afficher en fonction d'un id de diffusion donn  
/// </summary>
/// <returns> Liste des Message </returns>

public async Task<List<Message>> FindListMessageByDiffusionId(long diffusionId, CancellationToken cancellationToken)
{
    // Contr  les :
    ArgumentControlId(diffusionId);

    // Requête :
    List<Message> listMessage = await _dbSet.AsNoTracking()
        .Where(m => m.DiffusionId == diffusionId)
        .Include(m => m.Parametrage)
        .Select(m => new Message
        {
            Id = m.Id,
            Texte = m.Texte,
            Statut = m.Statut,
            CreatedAt = m.CreatedAt,
            DiffusionId = m.DiffusionId,
            ParametrageId = m.ParametrageId,
            Parametrage = m.Parametrage == null ? new() : new Parametrage
            {
                Id = m.Parametrage.Id,
                Duree = m.Parametrage.Duree
            }
        }).ToListAsync(cancellationToken);

    return listMessage;
}
```

Image 6 : Code de la partie DAO, permettant d'accéder aux données

d) Interfaces

Les interfaces `IMessageService` et `IMessageDao` permettent d'injecter ces composants proprement, facilitant les tests unitaires et le découplage entre couches.

3. Bilan

Cette semaine a permis de **structurer une fonctionnalité clé** du projet : le filtrage intelligent de messages actifs liés à une liste de diffusion.

Grâce à un découpage en **Controller** → **Service** → **DAO** → **Base**, la solution est claire, évolutive et testable.

Cela prépare le terrain pour d'éventuelles extensions comme l'ajout de filtres supplémentaires (par utilisateur, priorité, etc.) ou une pagination des résultats.

Semaine 4 :

L'objectif de cette semaine était de développer une **fonctionnalité permettant de modifier le statut d'un message**, afin d'assurer un suivi de consultation. Cette évolution s'intègre dans le flux de diffusion, en marquant les messages comme « **CONSULTÉS** » une fois qu'ils ont été récupérés ou affichés par l'interface client.

1. Fonctionnalité ciblée

Création d'un **endpoint HTTP PATCH** permettant de :

- Recevoir un **identifiant de message** (`idMessage`),
- Vérifier que ce message est bien dans le statut `EN_COURS`,
- Le mettre à jour avec le statut `CONSULTER` si c'est le cas,
- Sauvegarder le changement en base de données.

Ce mécanisme permet de **suivre la consultation réelle des messages** par les clients ou les systèmes connectés.

2. Étapes techniques de mise en œuvre

a) Création du contrôleur (endpoint API)

Un nouvel endpoint a été ajouté dans le contrôleur `MessageController`, avec une route de type `PUT`, permettant de déclencher l'action de mise à jour via l'URL.

```

/// <summary>
/// Mise à jour du statut du Message vers "CONSULTER"
/// </summary>
/// <returns> Id du Message modifié. </returns>

[HttpPut("/Statut/Consulter/{id}")]
public async Task<long> UpdateNotificationStatutToConsulterById([FromRoute, Required] long messageId, CancellationToken cancellationToken)
{
    // Verification de statut pour qu'il soit noté comme "en cours", refus si statut à pour valeur "a envoyé" ou "erreur"
    if (await _MessageServiceApplication.FindMessageStatutByMessageId(messageId, cancellationToken) == MessageStatutEnum.EN_COURS)
    {
        try
        {
            // Modification du statut vers consulté :
            await _MessageServiceApplication.UpdateNotificationStatutToConsulterById(messageId, cancellationToken, saveChanges: true);
        }
        catch (Exception exception)
        {
        }
    }

    return messageId;
}

```

Image 7 : Code de l'endpoint "UpdateNotificationStatutToConsulterById"

b) Logique métier dans le service

Dans la couche service, la méthode `UpdateNotificationStatutToConsulterById` contient :

- La récupération du message via le DAO,
- Une vérification du statut actuel,
- La mise à jour du statut vers `CONSULTER`,
- L'enregistrement de la modification.

Cette logique garantit que **seuls les messages encore "EN_COURS"** sont modifiables.

```

public async Task<long> UpdateNotificationStatutToConsulterById(long id, CancellationToken cancellationToken, bool saveChanges = true)
{
    try
    {
        await _uow.MessageDAO.UpdateNotificationStatutToConsulterById(id, cancellationToken, saveChanges: true);
        return id;
    }
    catch (Exception ex) { return id; }
}

```

Image 8 : Code du service permettant de lier la partie métier à la DAO

c) DAO : récupération et mise à jour

Le **DAO** (**MessageDao**) s'occupe de la lecture et de l'écriture dans la base, en s'appuyant sur Entity Framework.

Deux méthodes clés sont utilisées :

- **ExecuteUpdateAsync** : pour modifier le message,
- **saveChangesAsync** : pour appliquer la modification.

d) Utilisation des interfaces

Les interfaces **IMessageService** et **IMessageDao** ont été mises à jour pour intégrer cette nouvelle méthode, assurant un bon découplage et facilitant les tests unitaires.

Bilan

Cette fonctionnalité simple mais essentielle permet désormais de **suivre l'état d'un message après sa lecture**.

Elle s'inscrit dans une logique de traçabilité et d'évolution du statut, et prépare le terrain pour des usages plus avancés comme les statistiques de consultation, les relances ou la gestion du cycle de vie complet d'un message.

Semaine 5 :

Cette semaine a été consacrée à la mise en place des **contrôles (checks) de validité des notifications** et à l'automatisation de la mise à jour de leur statut. Ces vérifications garantissent que seules les notifications encore valides peuvent être consultées et que celles expirées sont correctement identifiées.

1. Mise en place des contrôles (checks)

Deux types de vérifications ont été ajoutés dans la logique métier :

Check du statut de la notification

- Lors de l'accès à une notification, le système vérifie qu'elle est bien en statut **EN_COURS**.
- Si ce n'est pas le cas, elle est considérée comme non valide et n'est pas renvoyée.

Check de la validité de la durée

- Chaque notification possède une **date de création** et un **délai de validité**.

- Lors des requêtes, le système calcule si la **date de création + délai** est supérieure à la date actuelle.
- Si le délai est dépassé, la notification est considérée comme expirée.

2. Mise à jour automatique du statut expiré

Pour garantir la cohérence des données, une logique a été développée pour **mettre à jour automatiquement les notifications expirées** :

- Lorsqu'une notification est consultée ou qu'une opération pertinente est réalisée, le système vérifie si sa durée est dépassée.
- Si c'est le cas, son statut est immédiatement mis à jour en **EXPIRER**.
- Cela évite qu'un message périmé puisse encore apparaître comme actif.

3. Exemple de logique technique

La vérification est faite dans le **service métier** par un tri :

```
/// <summary>
/// Prends une list de message et les trie en fonctions de leur validité. Resort tout les messages ayant encore une duree valide
/// </summary>
/// <param name="listMessage"> List des messages à vérifier</param>
/// <returns> List de message encore actif </returns>
public async Task<List<Message>> SortValiditeDuree(List<Message> listMessage, CancellationToken cancellationToken)
{
    DateTime now = DateTime.Now;

    List<Message> listMessageExpired = listMessage.Where(m => m.CreatedAt.Value.AddMinutes(m.Parametrage.Duree.Value) < now).ToList();
    List<Message> listMessageChecked = listMessage.Where(m => m.CreatedAt.Value.AddMinutes(m.Parametrage.Duree.Value) >= now).ToList();

    await UpdateNotificationStatutToExpireByListMessage(listMessageChecked, cancellationToken);

    return listMessageChecked;
}
```

Image 9 : Code du tri de la liste pour expirer

Ce tri ressort une liste de message avec un délai vérifié, et envoie tous les messages avec délai dépassé directement dans la mise à jour automatique :

```

/// <summary>
/// Mise à jour du statut du Message vers "EXPIRE"
/// </summary>
/// <param name="MessageId"> Identifiant du Message à modifier. </param>
/// <returns> true une fois les modifications apportées, false en cas de problème </returns>

public async Task<bool> UpdateNotificationStatutToExpireByListMessage(List<Message> listMessage, CancellationToken cancellationToken, bool saveChanges = true)
{
    try
    {
        for (int cpt = 0; cpt < listMessage.Count; cpt++)
        {
            await _dbSet.Where(m => m.Id == listMessage[cpt].Id).ExecuteUpdateAsync(m => m.SetProperty(m => m.Statut, MessageStatutEnum.EXPIRE), cancellationToken);
        }

        if (saveChanges) await _context.SaveChangesAsync(cancellationToken);
        return true;
    }
    catch (Exception ex)
    {
        return false;
    }
}

```

Image 10 : Code de la mise à jour automatique vers expire

Bilan

Grâce à ces évolutions, le projet dispose désormais :

- D'un **filtrage fiable des notifications valides**,
- D'un mécanisme automatique de mise à jour des statuts,
- D'une meilleure qualité et traçabilité des données.

Ces contrôles sont essentiels pour garantir que l'application ne traite que des messages pertinents et à jour.

Semaine 6 :

Cette dernière semaine a été consacrée à la création et à l'organisation de la **documentation** du projet afin de faciliter sa compréhension, sa prise en main et sa maintenance par d'autres développeurs ou utilisateurs.

1. Création du fichier **README.md**

Le fichier **README.md** a été rédigé pour centraliser toutes les informations essentielles :

- **Présentation du projet** (objectif, contexte, principales fonctionnalités)

- **Instructions d'installation** (prérequis, configuration Docker et SQL Server)
- **Démarrage rapide** (commandes pour lancer l'application)
- **Exemples d'utilisation des endpoints API**
- **Bonnes pratiques à suivre pour contribuer**

Ce fichier constitue le **point d'entrée principal** pour tout nouveau contributeur.

2. Documentation des endpoints API

Des sections dédiées ont été créées pour détailler :

- La liste complète des **endpoints disponibles** (GET, PATCH, etc.)
- Les paramètres attendus et les exemples de requêtes
- Les règles métier appliquées (vérification des statuts, gestion des délais, etc.)
- Les réponses possibles et les codes HTTP associés

Cette partie aide à comprendre le comportement de l'API sans avoir à parcourir tout le code.

3. Explications sur la structure technique

Des explications ont été rédigées sur :

- La **structure en couches** (Controller, Service, DAO)
- Les **interfaces** utilisées pour injecter les dépendances
- Le fonctionnement des **migrations Entity Framework**
- Les bonnes pratiques de mise à jour du schéma

Bilan

La mise en place de cette documentation permet au projet d'être **clair, maintenable et facilement transférable**, tout en servant de référence unique pour les évolutions futures.

Conclusion :

Ce stage réalisé au sein d'Actemium Process Control a été une expérience extrêmement formatrice et stimulante, tant sur le plan technique que sur le plan humain. En rejoignant cette équipe, j'ai eu l'opportunité de travailler dans un environnement professionnel exigeant, où l'autonomie et la rigueur occupent une place centrale.

Dès les premières semaines, j'ai été accompagné de façon bienveillante et encadré par des professionnels expérimentés, qui ont su me guider et me faire progresser tout en me laissant la liberté de chercher, d'expérimenter et de proposer des solutions. Cette confiance m'a permis de m'impliquer pleinement dans le développement du module de notifications, qui constituait un enjeu important pour la fiabilité et la traçabilité des opérations dans le logiciel GPAC.

Sur le plan technique, j'ai consolidé mes compétences en C#, en conception de base de données sous SQL Server, et en utilisation de technologies modernes comme Docker et Entity Framework. J'ai pu appliquer une architecture claire en couches (Controller, Service, DAO) et comprendre concrètement comment organiser un projet de façon professionnelle, tout en garantissant sa maintenabilité et sa qualité. La mise en place des contrôles de validité, la gestion des statuts, et la création de la documentation détaillée ont été autant d'étapes qui m'ont permis de mesurer l'importance d'un code fiable, compréhensible et bien documenté.

Au-delà des aspects techniques, ce stage m'a appris à travailler de manière organisée et méthodique, à respecter des délais et des priorités, et à communiquer efficacement avec mes collègues. J'ai pris conscience de l'importance des échanges réguliers, de la relecture de code et du partage d'informations au sein d'une équipe.

Je ressors de cette expérience avec la conviction renforcée de vouloir poursuivre dans le domaine du développement logiciel, et avec la satisfaction d'avoir contribué, même modestement, à un projet concret utilisé dans un contexte industriel. Cette première immersion m'a donné confiance dans mes capacités et m'a permis d'acquérir des bases solides qui me seront précieuses pour la suite de mon parcours professionnel et mes études à venir.